

151. À toi : ajouter ta propre fonction de la console, pas à pas

La marche à suivre complète, de l'idée à « `#include <ta_fonction>` » — et un cadre dessiné avec `bande()` et `pile()`.

1. Ce qu'il faut comprendre

On revient à la `bande()` de la console (`outils.cpp` ne contient plus que `pile()`, comme au 15.6), et on s'en sert pour **un cadre** : deux bandes (le haut et le bas), deux piles (la gauche et la droite). C'est la seule nouveauté du programme.

Déroulé du cadre : `bande(3, 3, ..., 14)` → le haut, colonnes 3 à 16 ; `bande(3, 12, ..., 14)` → le bas ; `pile(3, 4, ..., 8)` → la gauche, lignes 4 à 11 ; `pile(16, 4, ..., 8)` → la droite. Les coins appartiennent aux bandes : les piles commencent une ligne plus bas.

Et maintenant, pour faire entrer `pile()` dans la console à son tour, voici la marche à suivre — la même pour n'importe quelle fonction :

Étape 1 — l'écrire et l'essayer dans ton programme. D'abord dans `principal.cpp`, ou dans un fichier voisin comme `outils.cpp`. Tant qu'elle n'est pas parfaite, elle reste là : c'est plus facile à corriger.

Étape 2 — choisir son nom. Un nom qu'aucune fonction de la console ne porte déjà (la liste est dans `compilateur/inclusion.js`, tableau `BIBLIOTHEQUES`). Le nom du `#include` sera exactement celui de la fonction : `pile` → `#include <pile>`.

Étape 3 — `compilateur/emetteur.js`. Cherche `const FONCTIONS_EN_C`. Juste au-dessus, colle ta fonction dans une constante, `const SOURCE_PILE = ...`, entre deux accents graves (AltGr + 7), comme `SOURCE_BANDE` juste à côté. Dans le tableau, ajoute `pile: SOURCE_PILE,`. Retire son `#include <poser>` : la console n'en a pas besoin.

Étape 4 — `compilateur/inclusion.js`. Dans `BIBLIOTHEQUES`, ajoute une ligne : `pile: 'pose la même tuile plusieurs fois, vers le bas',`. Sans elle, `#include <pile>` serait refusé : « je ne connais pas cette bibliothèque ».

Étape 5 — `aide-fonctions.js`. Dans `FONCTIONS`, ajoute `pile: { args: ['colonne', 'ligne', 'tuile', 'hauteur'], dit: '...' },` pour que l'éditeur la propose.

Étape 6 — essayer. Recharge la page (**Ctrl+F5**), enlève `pile()` d'`outils.cpp`, écris `#include <pile>` dans `principal.cpp`, et lance. En ligne de commande : `node outils/gb3.mjs mon-essai.cpp` compile un fichier, et `npm run verifier` vérifie que rien d'autre n'est cassé.

Étape 7 — sa leçon. Dans ce projet, chaque `#include` a son tuto : une leçon dans `tuto/lecons.js` dont le programme commence par `// ---- #include <pile> : ...`. Le parcours la place tout seul juste avant la première étape qui emploie `pile()`. Copie celle de `bande()` et change ce qu'il faut.

Pour aller plus loin : les fonctions qui parlent directement au matériel (l'écran, le son) ne sont pas écrites en C mais en instructions du processeur, dans `compilateur/emetteur.js` (cherche `if (nom === 'cacherPanneau')`). C'est plus difficile : la façon en C suffit pour tout ce qu'on peut écrire avec les fonctions existantes.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <bande>` → [148.1. La fonction bande\(\) — la même tuile, plusieurs fois](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1010 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage	– la mise en route de la console
bande()	38 o	← main()	– une fonction fournie par le langage
pile()	42 o	← main()	– ta fonction
main()	83 o	← démarrage	– ta fonction
VBlank	8 o	← la console, à chaque image	– compte les images, 60 fois par seconde
AttendreImage	28 o	← image()	– attend l'image suivante
AttendreVBlank	16 o	← démarrage	– attend que l'écran soit libre
AdresseCase	10 o	← poser()	– trouve l'adresse d'une case de l'écran
CopierTuiles	18 o	← démarrage	– copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage	– vide les deux cartes de l'écran
Tuiles	704 o	← CopierTuiles	– la police entière

Lettres gravées : toutes (poser() reçoit un numéro calculé)

Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, EffacerCases, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

poser()	× 2	38 o sur place, dans bande(), pile()	+ AdresseCase (10 o)
image()	× 1	3 o sur place, dans main()	+ AttendreImage (28 o)

Tes fonctions appelées :

bande()	× 2	38 o sur place, dans main()
pile()	× 2	38 o sur place, dans main()

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// Un cadre de X : bande() (de la console) pour le haut et le bas,
// pile() (d'outils.cpp) pour les côtés.
//
//   colonnes 3 à 16, lignes 3 à 12 :
//
//   XXXXXXXXXXXXXXXX ← bande(3, 3, X, 14)   le haut
//   X                X ← pile(3, 4, X, 8) et pile(16, 4, X, 8)
//   X                X   les côtés, lignes 4 à 11
//   XXXXXXXXXXXXXXXX ← bande(3, 12, X, 14)  le bas

#include <ALPHABET>
#include <bande>      // pose la même tuile plusieurs fois, de gauche à droite
#include "outils.cpp" // pile()
```

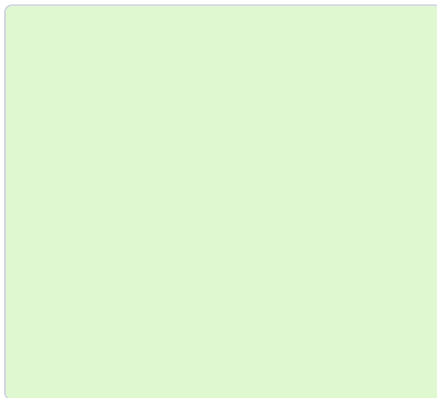
2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    bande(3, 3, ALPHABET[23], 14);    // le haut  (ALPHABET[23] : le X, 24e lettre)
    bande(3, 12, ALPHABET[23], 14);   // le bas
    pile(3, 4, ALPHABET[23], 8);      // le côté gauche : lignes 4 à 11
    pile(16, 4, ALPHABET[23], 8);     // le côté droit

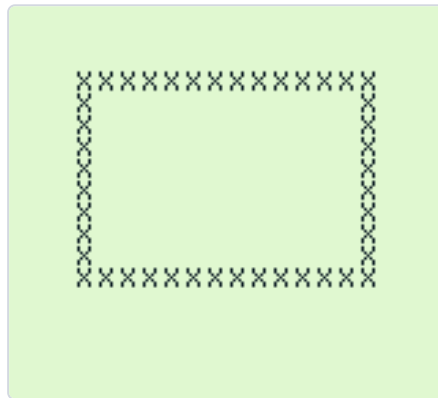
    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Un cadre de X au milieu de l'écran : colonnes 3 à 16, lignes 3 à 12, vide à l'intérieur.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le haut : quatorze X
- ✓ le bas : quatorze X
- ✓ les côtés : un X à chaque bout, rien entre les deux

Ensuite : **152. Un raccourci à toi : écrire `poserDevant()` soi-même** — chapitre 25 : Tes propres `#include` : ajouter une fonction. · gameboy3 — écrire une cartouche Game Boy en C++